

REST 練習 8：REST Resource 的可測試性

Lecture

rs01~rs07 一路把商業邏輯直接寫在 `IF_REST_RESOURCE~GET/PUT/DELETE` 這幾個方法裡——這樣寫得快，但有一個結構性問題：這幾個方法的簽章綁死了 **REST** 框架的型別。`GET/DELETE` 沒有參數，邏輯要靠讀 `mo_request` (`IF_REST_REQUEST` 參考) 才能拿到路徑參數；`PUT/POST` 的 `IO_ENTITY` 是 `IF_REST_ENTITY` 參考；回應要透過 `mo_response` (`IF_REST_RESPONSE` 參考) 設定狀態碼跟寫回應內容。這些物件全部是 `CL_REST_HTTP_HANDLER~DO_HANDLE` 在真正收到一個 HTTP request 的時候才會建立、注入進 Resource 物件——如果想幫 rs07 的 `ZCL_RS07_FLIGHT~PUT` 寫一個 ABAP Unit test，得先自己生出一個假的 `IF_REST_ENTITY` 物件才能呼叫得動這個方法，而 `mo_request/mo_response` 又是父類別 `CL_REST_RESOURCE` 的 `protected` 屬性，不是建構子注入的，測試類別要塞值進去更加麻煩。結果就是：這幾題的程式碼完全沒辦法直接寫 **ABAP Unit test**，只能靠掛 SICF、用 `SPROX_HTTP_REQUEST` 手動打 HTTP 才能驗證邏輯對不對——rs07 那個「`fldate` 格式錯誤沒被擋下來」的 bug，就是一路寫到掛 SICF 實測才被抓到，如果早一步能寫測試，這個 bug 在開發階段就會被抓出來，不用等到手動測試那一步。

解法呼應 op10「為可測試而設計」跟 op12「取數薄、計算純」這兩堂課教過的東西，只是套用在 REST 的情境：****把商業邏輯整個搬出 Resource 類別，搬進一個完全不依賴 `IF_REST_*` 型別的普通 ABAP 類別 (這題叫 `ZCL_RS08_FLIGHT_SERVICE`)。這個 Service 類別的方法簽章全部是 ABAP 原生型別 (`STRING`、自訂結構)**，跟「這是不是一個 HTTP 請求」完全無關——**ABAP Unit 的測試類別可以直接 `zcl_rs08_flight_service=>parse_keys( ... )` 呼叫、餵字串、斷言結果，不用碰任何 REST 框架的東西。Resource 類別 (`ZCL_RS08_FLIGHT`) 瘦身成薄薄一層轉接****：從 `mo_request` 讀出原始字串、丟給 Service 處理、把 Service 回傳的結果或拋出的例外轉成 HTTP 回應——這層轉接邏輯本身很薄、很難寫錯，不寫測試也還算安全 (呼應 op12「取數薄、計算純」裡「薄到幾乎不會錯」的說法，只是這裡「薄」的是 REST 轉接層，op12 薄的是 DB 讀取層，道理相通)。

這題也延續了 op12「取數/計算」拆分的精神，只是拆分的軸線不是「DB vs 純計算」而是「REST 框架 vs 純邏輯」：Service 類別裡 `parse_keys` (驗證路徑參數格式) 跟 `validate_update` (驗證 Body 必填欄位) 完全不碰資料庫，是純邏輯，這題會幫它們寫滿 ABAP Unit test；`get_flight/update_flight/delete_flight` 會真的碰 `SFLIGHT` 資料表，依照 op10 思考題 3 跟 op12 立下的課程慣例，這類方法故意保持「薄」(邏輯簡單到用看的就能確認對不對)，不強求寫 ABAP Unit test。

學習目標

- 理解為什麼 REST Resource 類別的 `GET/PUT/DELETE` 方法很難直接寫 ABAP Unit test：方法簽章綁死 `IF_REST_ENTITY/mo_request/mo_response` 這些框架型別，要測就得先假造一整套 REST 框架物件
- 學會把商業邏輯搬進一個不依賴任何 REST 框架型別的 Service 類別，讓 Resource 類別瘦身成「解析 request → 呼叫 Service → 組 response」的薄層轉接
- 認得這題為什麼用 `CLASS-METHODS` (靜態方法) 而不是 op10 `ZCL_0010_QUOTE` 那種「建構子注入依賴」的實例方法設計——op10 的 `calc_quote` 需要注入可替換的折扣策略物件 (`ZIF_0007_DISCOUNT`)，這裡的 `ZCL_RS08_FLIGHT_SERVICE` 完全沒有這種「可替換的協作物件」需要注入，用靜態方法呼叫端連 `NEW` 都不用，更直接
- 延續 op12「取數薄、計算純」的分層原則，只是這次拆分的軸線是「REST 框架 vs 純邏輯」而不是「DB vs 純計算」：`parse_keys/validate_update` 兩個純邏輯方法全部寫 ABAP Unit test；

`get_flight/update_flight/delete_flight` 這幾個會碰資料庫的方法，依課程慣例保持薄、不強求寫測試

- 體會「重構不改行為」（呼應 op12 期末重構的精神）：rs08 對外的 HTTP 行為應該跟 rs07 一模一樣（同樣的 URL、同樣的狀態碼、同樣的 JSON 格式），改變的只是程式碼內部的分層方式

事前準備

ADT 端物件已由課程準備好（\$TMP）：

- `ZCX_RS08_FLIGHT_ERROR`——業務例外，跟 rs07 的 `ZCX_RS07_FLIGHT_ERROR` 設計完全一樣（`mv_status_code/mv_error_code/mv_message`）
- `ZCL_RS08_FLIGHT_SERVICE`——純 ABAP 商業邏輯層，不依賴任何 REST 框架型別，附 **ABAP Unit** 測試（ADT 的 Test Classes 頁籤）
- `ZCL_RS08_APP`——Application Class，router 掛 `/flights/{carrid}/{connid}/{fldate}` → `ZCL_RS08_FLIGHT`；覆寫 `HANDLE_CSRF_TOKEN` 跳過 CSRF 檢查（比照 rs06/rs07）
- `ZCL_RS08_FLIGHT`——薄層 Resource，`GET/PUT/DELETE` 都只做「解析 → 呼叫 Service → 組回應」

題目需求（對照已建好的答案物件）

1. `ZCL_RS08_FLIGHT_SERVICE`：公開三個型別（`ty_keys/ty_flight/ty_update`，讓 Resource 可以用 `zcl_rs08_flight_service=>ty_xxx` 引用）跟五個靜態方法：

```
``abap CLASS zcl_rs08_flight_service DEFINITION PUBLIC FINAL CREATE PUBLIC. PUBLIC SECTION. TYPES: BEGIN OF ty_keys, carrid TYPE s_carr_id, connid TYPE s_conn_id, fldate TYPE s_date, END OF ty_keys, BEGIN OF ty_flight, carrid TYPE s_carr_id, connid TYPE s_conn_id, fldate TYPE s_date, price TYPE s_price, currency TYPE s_currcode, END OF ty_flight, BEGIN OF ty_update, price TYPE s_price, currency TYPE s_currcode, END OF ty_update.
```

```
CLASS-METHODS: parse_keys IMPORTING iv_carrid TYPE string iv_connid TYPE string iv_fldate TYPE string RETURNING VALUE(rs_keys) TYPE ty_keys RAISING zcx_rs08_flight_error, validate_update IMPORTING is_update TYPE ty_update RAISING zcx_rs08_flight_error, get_flight IMPORTING is_keys TYPE ty_keys RETURNING VALUE(rs_flight) TYPE ty_flight RAISING zcx_rs08_flight_error, update_flight IMPORTING is_keys TYPE ty_keys is_update TYPE ty_update RETURNING VALUE(rs_flight) TYPE ty_flight RAISING zcx_rs08_flight_error, delete_flight IMPORTING is_keys TYPE ty_keys RAISING zcx_rs08_flight_error. ENDCLASS.``
```

`parse_keys`（純邏輯，就是 rs07 修正過的格式驗證，只是搬進了一個不依賴 REST 框架的方法）：

```
``abap METHOD parse_keys. IF strlen( iv_fldate ) <> 8 OR iv_fldate CN '0123456789'. RAISE EXCEPTION TYPE zcx_rs08_flight_error EXPORTING iv_status_code = cl_rest_status_code=>gc_client_error_bad_request iv_error_code = 'INVALID_DATE' iv_message = |fldate 格式不正確，必須是 8 碼數字（YYYYMMDD）|. ENDIF. rs_keys-carrid = iv_carrid. rs_keys-connid = iv_connid. rs_keys-fldate = iv_fldate. ENDMETHOD.``
```

`validate_update`（純邏輯，跟 rs07 的必填檢查一樣）、`get_flight/update_flight/delete_flight`（會碰資料庫，邏輯跟 rs07 完全一樣，只是搬到 Service 類別）細節不重複列出，見 `zcl_rs08_flight_service.clas.abap`。

1. Test Classes 頁籤：兩個測試類別，只測 `parse_keys/validate_update` 這兩個純邏輯方法：

```
``abap CLASS ltc_parse_keys DEFINITION FINAL FOR TESTING RISK LEVEL HARMLESS DURATION
SHORT. PRIVATE SECTION. METHODS: valid_input_passes FOR TESTING RAISING
zcx_rs08_flight_error, fldate_wrong_length_raises FOR TESTING, fldate_non_numeric_raises
FOR TESTING. ENDCLASS. ``
```

`fldate_non_numeric_raises` 這個方法就是直接複製 **rs07** 那個真實踩到的 **bug** 場景 (`iv_fldate = 'abcd'`) · 斷言一定要拋出 `zcx_rs08_flight_error` (`errorCode = INVALID_DATE`) :

```
``abap METHOD fldate_non_numeric_raises. DATA(lv_raised) = abap_false.
```

```
TRY. zcl_rs08_flight_service=>parse_keys( iv_carrid = 'LH' iv_connid = '0400' iv_fldate = 'abcd' ). CATCH
zcx_rs08_flight_error INTO DATA(lx_error). lv_raised = abap_true. cl_abap_unit_assert=>assert_equals( act =
lx_error->mv_error_code exp = 'INVALID_DATE' msg = '非數字字元應該回 INVALID_DATE' ). ENDTRY.
```

```
cl_abap_unit_assert=>assert_true( act = lv_raised msg = '應該要拋出 zcx_rs08_flight_error' ). ENDMETHOD. ``
```

`ltc_validate_update` 三個方法 (合法輸入不拋例外、缺 `price`、缺 `currency`) 邏輯是同一套模式，見 `zcl_rs08_flight_service.clas.testclasses.abap`。這套系統的 **CL_ABAP_UNIT_ASSERT** 沒有 `fail()`/`assert_exception()` 這兩個較新版本才有的輔助方法 (查過原始碼確認過，只有 `assert_bound/assert_char_cp/assert_differs/assert_equals/assert_equals_float/assert_false/assert_initial/assert_true` 等)，所以「斷言一定會拋出例外」這件事用「**TRY...CATCH** 裡把旗標設成 `abap_true`，**TRY** 區塊外再 `assert_true` 這個旗標」的傳統寫法達成，不是用一行 `assert_exception` 打發。

1. **ZCL_RS08_APP / ZCL_RS08_FLIGHT**：路由與 **CSRF** 跳過寫法跟 **rs07** 一模一樣，**GET/PUT/DELETE** 改成呼叫 **Service** (節錄 **GET**)：

```
``abap METHOD if_rest_resource~get. TRY. DATA(ls_keys) = zcl_rs08_flight_service=>parse_keys( iv_carrid =
mo_request->get_uri_attribute( 'carrid' ) iv_connid = mo_request->get_uri_attribute( 'connid' ) iv_fldate =
mo_request->get_uri_attribute( 'fldate' ) ).
```

```
DATA(ls_flight) = zcl_rs08_flight_service=>get_flight( ls_keys ). render_flight( ls_flight ). CATCH
zcx_rs08_flight_error INTO DATA(lx_error). render_error( lx_error ). CATCH cx_root INTO DATA(lx_unexpected).
render_error( wrap_error( lx_unexpected ) ). ENDTRY. ENDMETHOD. ``
```

`render_flight/render_error/wrap_error` 這三個組回應的私有方法還是留在 **Resource** 類別，不搬去 **Service**——因為它們依賴 `mo_response` (**IF_REST_RESPONSE**)，本質上就是 **REST** 專屬的轉接邏輯，搬過去反而違背了「**Service** 不依賴 **REST** 型別」的初衷。**PUT/DELETE** 完整程式碼見 `zcl_rs08_flight.clas.abap`。

SICF 掛載步驟 (比照 rs01~rs07)

沿用既有的 `/sap/bc/zrest_training` 分類節點，這題只要在其底下再加一個子節點：

1. SICF 交易碼，在 `zrest_training` node 上右鍵 → **New Sub-Element**，Service Name 填 **rs08**
2. Handler List 掛 **ZCL_RS08_APP**
3. Activate Service
4. 用 **SPROX_HTTP_REQUEST** 測試——這題對外的行為應該跟 **rs07** 一模一樣 (重構不改行為)，可以直接照抄 **rs07** 的測試步驟，只把路徑從 **rs07** 換成 **rs08**：

- GET/PUT 成功、PUT 缺欄位 (400)、查無資源 (404)、fldate 格式錯誤 (400) ——測試資料一樣可以用 LH/400/20190104 (如果還沒被 rs07 的 PUT 測試改過，起始 price 是 666.00 EUR；如果 rs07 已經改成 680.00，這題就從 680.00 繼續往下測，不影響驗證邏輯本身)
- DELETE 一樣受限於 SPROX_HTTP_REQUEST 沒有 DELETE 選項，跟 rs07 一樣待補測

預期輸出 (範例)

跟 rs07 完全一樣 (同一組狀態碼、同一種 JSON 格式)，這裡不重複列出，差異只在如何驗證邏輯本身是對的：這題除了掛 SICF 手動測，還多了一組可以隨時重跑的 ABAP Unit test。

ADT 執行 Ctrl+Shift+F10 (或用 sap_run_unit_test)：

```
ZCL_RS08_FLIGHT_SERVICE > LTC_PARSE_KEYS
  FLDATE_NON_NUMERIC_RAISES    passed
  FLDATE_WRONG_LENGTH_RAISES   passed
  VALID_INPUT_PASSES           passed
ZCL_RS08_FLIGHT_SERVICE > LTC_VALIDATE_UPDATE
  MISSING_CURRENCY_RAISES      passed
  MISSING_PRICE_RAISES         passed
  VALID_INPUT_PASSES           passed
```

團隊實務備註

- 這題的六個測試全部不碰資料庫、不掛 SICF、不需要 SAP GUI 也能跑 (sap_run_unit_test 這個 MCP 工具就能直接觸發並拿到 JSON 結果) ——這是把邏輯搬出 REST 框架之後拿到的實際好處：驗證商業邏輯的迴圈從「改程式 → 部署 → 掛 SICF → 開 SAP GUI → 打 HTTP request → 看回應」大幅縮短成「改程式 → 跑測試 → 看紅綠」
- 如果 rs07 當初就用這種分層寫法，fldate 格式錯誤那個 bug 會在寫完 parse_keys 的當下就被測試抓到，根本不用等到掛 SICF 用 SPROX_HTTP_REQUEST 手動測才發現——這是這題想傳達的核心價值，不是「多寫幾個類別」這種形式上的重構
- get_flight/update_flight/delete_flight 沒有寫測試，不代表這些方法不重要或可以隨便寫，而是課程刻意的範圍決策 (呼應 op10 思考題 3 跟 op12 的「取數薄」)：這幾個方法邏輯簡單到「看程式碼就能確認對不對」，真的要測需要引入資料庫測試替身 (test double) 或整合測試的技巧，超出這題範圍，見思考題 2
- 靜態方法 (CLASS-METHODS) 沒有狀態，天生就是 thread-safe、呼叫端不用管理物件生命週期——但也因為沒有物件，沒辦法用 op10 那種「建構子注入不同實作」的方式替換行為；這題不需要替換行為 (SFLIGHT 就是唯一的資料來源，沒有「策略」可言)，所以靜態方法是合理選擇，不是偷懶

思考題

1. 如果之後想讓 get_flight 也能被 ABAP Unit 測試 (不碰真實 SFLIGHT 資料)，要怎麼改這個 Service 類別的設計？ (提示：回顧 op10「依賴用建構子注入」——如果把「怎麼從資料庫撈資料」抽成一個介面，ZCL_RS08_FLIGHT_SERVICE 改成實例方法、建構子注入這個介面的實作，測試時就可以注入一個回傳假資料的測試替身，這正是 rs08 目前沒做、但完全可以延伸的方向)
2. parse_keys/validate_update 這兩個方法即使 SFLIGHT 資料表被清空、或整個系統斷線，測試都應該照樣綠燈通過——為什麼？這跟 rs05 的 ?carrid=AA&fromdate=20991231 這種「要真的連資料庫查詢」的情境有什麼本質上的差異？

3. 這題把 `render_flight/render_error/wrap_error` 留在 Resource 類別、不搬進 Service——如果哪天要求「除了 REST，也要開放一個 RFC 介面呼叫同一套航班查詢/更新邏輯」，現在這個分層方式能不能直接重用 `ZCL_RS08_FLIGHT_SERVICE`？需要多寫什麼？

答案

見 `zcx_rs08_flight_error.clas.abap`、`zcl_rs08_flight_service.clas.abap` (+ `.testclasses.abap`)、`zcl_rs08_app.clas.abap`、`zcl_rs08_flight.clas.abap` (SAP 端物件 `ZCX_RS08_FLIGHT_ERROR` / `ZCL_RS08_FLIGHT_SERVICE` / `ZCL_RS08_APP` / `ZCL_RS08_FLIGHT`)。SICF Service 路徑 `/sap/bc/zrest_training/rs08`。